Computer Architecture Lab

(EL-2012)

LABORATORY REPORT

Fall 2026



(Lab#07)

Arithmetic Logic Unit

Student Name: Huzaifa Kashif Roll No: 24I-6501 Section: CE-A

Date performed: 6th September, 2026

MARKS AWARDED: _____ / 10

Signature and date of Instructor: _____
Lab Instructors: Dr. Nasir Ali Shah, Engr. Fasih Ahmad

Task 1:

Description:
We had to make a 1 bit ALU at 3 levels: behavioral, gate and data flow. A testbench has also been included to see the output. An according logisim circuit was also made.

Code:

Behavioral Level:

```
module alu(a, b, Cin, binv, select, result, CarOUT);
input a, b, Cin, binv;
input [1:0] select;
output reg result,CarOUT;
reg m;
reg sum;
reg cout;
always @ *
begin
m = b ^ binv;
{cout, sum} = a + m + Cin;
CarOUT = 0;


if (select == 2'b00)
   result = a & b;
else if (select == 2'b01)
   result = a | b;
else if (select == 2'b10) begin
   result = sum;
   CarOUT = cout;
end
else
   result = 0;
end
endmodule


module testbenchalu();
reg a, b, carryin, binv;
reg [1:0] select;
wire result, CarOUT;
```

```
alu uut(a, b, carryin, binv, select, result, CarOUT);

initial
begin
// AND
select = 2'b00;
a = 0; b = 1; carryin = 0; binv = 0;
#5 a = 1; b = 1;

// OR
#5 select = 2'b01;
a = 0; b = 0;
#5 a = 1; b = 0;

// Addition
#5 select = 2'b10;
binv = 0; carryin = 0;
a = 0; b = 1;
#5 a = 1; b = 1;

// Subtraction
#5 binv = 1; carryin = 1;
a = 0; b = 1;
#5 a = 1; b = 0;
// Default case
#5 select = 2'b11;
end
endmodule
```

Gate Flow:

```
module alugateflow(a, b, carryin, binv, select, result, carryout);

input a, b;
input carryin, binv;
input [1:0] select;
output result;
output carryout;

wire m;
wire x1, sum;
wire c1, c2, cout;

wire andoutput, oroutput;
wire andselect, orselect, operatorselect;
```

```
xor (m, b, binv);
xor (x1, a, m);
xor (sum, x1, carryin);

and (c1, a, m);
and (c2, x1, carryin);
or (cout, c1, c2);

and (andoutput, a, b);
or (oroutput, a, b);

wire n1, n0;
not (n1, select[1]);
not (n0, select[0]);

and (andselect, n1, n0);
and (orselect, n1, select[0]);
and (operatorselect, select[1], n0);

wire r1, r2, r3;

and (r1, andselect, andoutput);
and (r2, orselect, oroutput);
and (r3, operatorselect, sum);

or (result, r1, r2, r3);
and (carryout, operatorselect, cout);

endmodule



module testbenchalu();
reg a, b, carryin, binv;
reg [1:0] select;
wire result, CarOUT;

alugateflow uut(a, b, carryin, binv, select, result, CarOUT);

initial
begin
// AND
select = 2'b00;
a = 0; b = 1; carryin = 0; binv = 0;
#5 a = 1; b = 1;
```

```
// OR
#5 select = 2'b01;
a = 0; b = 0;
#5 a = 1; b = 0;

// Addition
#5 select = 2'b10;
binv = 0; carryin = 0;
a = 0; b = 1;
#5 a = 1; b = 1;

// Subtraction
#5 binv = 1; carryin = 1;
a = 0; b = 1;
#5 a = 1; b = 0;
// Default case
#5 select = 2'b11;
end
endmodule


Data Flow:
module aludataflow(a, b, carryin, binv, select, result, carryout);
input a, b;
input carryin, binv;
input [1:0] select;
output result;
output carryout;
wire m;
wire x1, sum;
wire c1, c2, cout;

wire andoutput, oroutput;
wire andselect, orselect, operatorselect;
assign m = b ^ binv;
assign x1 = a ^ m;
assign sum = x1 ^ carryin;
assign c1 = a & m;
assign c2 = x1 & carryin;
assign cout = c1 | c2;
assign andoutput = a & b;
assign oroutput = a | b;
assign andselect = ~select[1] & ~select[0];
assign orselect = ~select[1] & select[0];
assign operatorselect = select[1] & ~select[0];
assign result = (andselect & andoutput) |
```

```
(orselect & oroutput) |
(operatorselect & sum);
assign carryout = operatorselect & cout;
endmodule


module testbenchalu();
reg a, b, carryin, binv;
reg [1:0] select;
wire result, CarOUT;

aludataflow uut(a, b, carryin, binv, select, result, CarOUT);

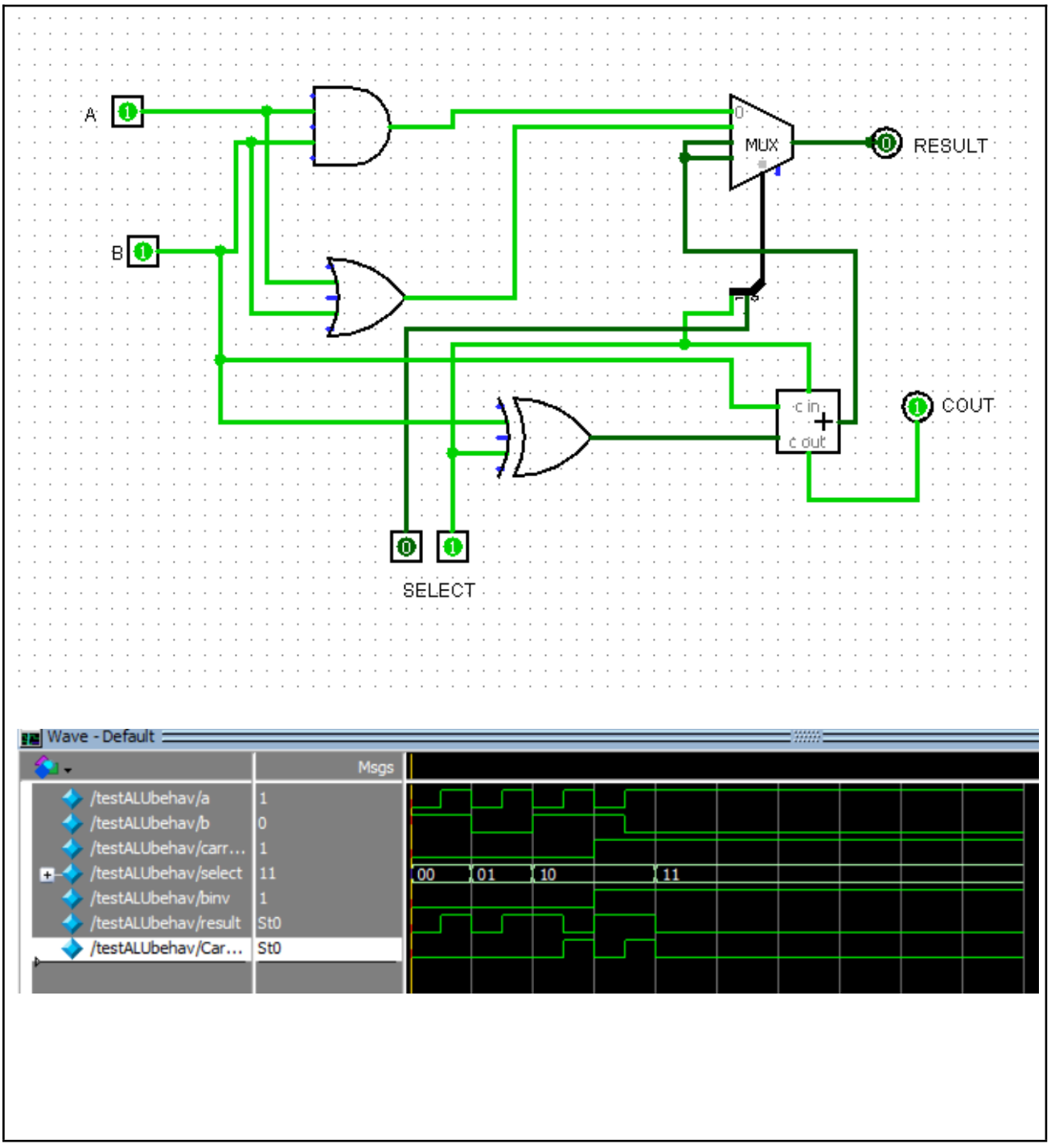
initial
begin
// AND
select = 2'b00;
a = 0; b = 1; carryin = 0; binv = 0;
#5 a = 1; b = 1;

// OR
#5 select = 2'b01;
a = 0; b = 0;
#5 a = 1; b = 0;

// Addition
#5 select = 2'b10;
binv = 0; carryin = 0;
a = 0; b = 1;
#5 a = 1; b = 1;

// Subtraction
#5 binv = 1; carryin = 1;
a = 0; b = 1;
#5 a = 1; b = 0;
// Default case
#5 select = 2'b11;
end
endmodule
```

Output:


A
B
MUX
RESULT
COUT
c in
c out
SELECT
Wave - Default
Msgs
/testALUbehav/a
/testALUbehav/b
/testALUbehav/carr...
/testALUbehav/select
/testALUbehav/binv
/testALUbehav/result
/testALUbehav/Car...
1
0
1
11
1
St0
St0
00
01
10
11

Task 2:

Description:
In this task a 4 bit ALU was created by cascading 4, 1 bit ALUs.

Code:

```
module alu1bit(a, b, Cin, binv, less, select, result, CarOUT);

input a, b, Cin, binv, less;
input [1:0] select;
output reg result, CarOUT;

reg m;
reg sum;
reg cout;

always @*
begin
   m = b ^ binv;
   {cout, sum} = a + m + Cin;
   CarOUT = 0;

   if (select == 2'b00)
      result = a & b;
   else if (select == 2'b01)
      result = a | b;
   else if (select == 2'b10) begin
      result = sum;
      CarOUT = cout;
   end
   else
      result = less;
end

endmodule


module alu4bit(a, b, op, result, CF, SF, ZF, OF);

input [3:0] a, b;
```

```
input [2:0] op;
output reg [3:0] result;
output reg CF, SF, ZF, OF;

wire binv;
wire [4:0] carry;
wire [3:0] tempresult;
wire lessbit;
wire overflow;
wire [3:0] subresult;
wire [4:0] subcarry;
wire suboverflow;
reg [1:0] select;

always @*
begin
   if (op == 3'b000)
      select = 2'b00;
   else if (op == 3'b001)
      select = 2'b01;
   else if (op == 3'b111)
      select = 2'b11;
   else
      select = 2'b10;
end

assign binv = (op == 3'b110 || op == 3'b111);
assign carry[0] = binv;

alu1bit bit0(a[0], b[0], carry[0], binv, lessbit,
         select, tempresult[0], carry[1]);

alu1bit bit1(a[1], b[1], carry[1], binv, 1'b0,
         select, tempresult[1], carry[2]);

alu1bit bit2(a[2], b[2], carry[2], binv, 1'b0,
         select, tempresult[2], carry[3]);

alu1bit bit3(a[3], b[3], carry[3], binv, 1'b0,
         select, tempresult[3], carry[4]);

// Independent subtraction for signed SLT
assign subcarry[0] = 1'b1;
assign subcarry[1] = a[0] ^ b[0] ? a[0] : subcarry[0];
assign subresult = a - b;
assign suboverflow = (a[3] ^ b[3]) & (a[3] ^ subresult[3]);
```

```
assign lessbit = subresult[3] ^ suboverflow;

assign overflow = carry[3] ^ carry[4];

always @*
begin
   result = tempresult;
   CF = 0;
   SF = result[3];
   ZF = (result == 4'b0000);
   OF = 0;

   if (op == 3'b010 || op == 3'b110)
   begin
      CF = carry[4];
      OF = overflow;
   end
end

endmodule

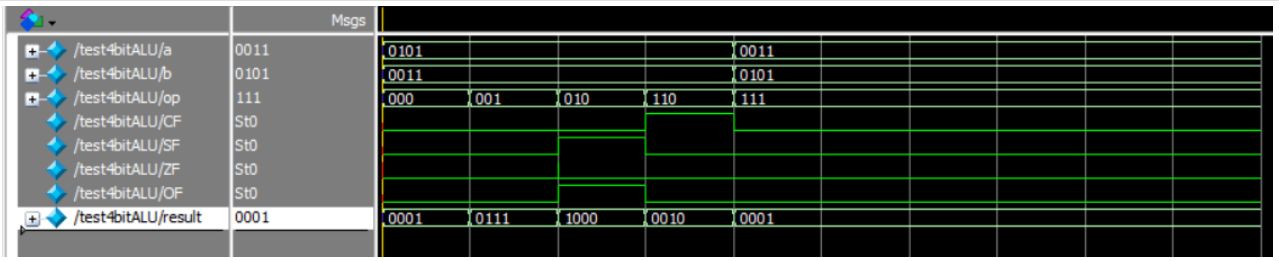

module test4bitALU();

reg [3:0] a, b;
reg [2:0] op;
wire [3:0] result;
wire CF, SF, ZF, OF;

alu4bit uut(a, b, op, result, CF, SF, ZF, OF);

initial
begin
   a = 5; b = 3; op = 3'b000; // and
   #10 a = 5; b = 3; op = 3'b001; // or
   #10 a = 5; b = 3; op = 3'b010; // add
   #10 a = 5; b = 3; op = 3'b110; // sub
   #10 a = 3; b = 5; op = 3'b111; // SLT
end
endmodule
```

Output:



Task 3:

Description:

Here the ALU is parameterized so that any N type bit can be inputted making the ALU more generalized.

Code:

```
module aluParameter #(parameter N = 4) (
   input [N-1:0] a,
   input [N-1:0] b,
   input [2:0] select,
   output reg [N-1:0] result
);

always @*
begin
   if (select == 3'b000)
      result = a + b;  // add
   else if (select == 3'b001)
      result = a - b;  // sub
   else if (select == 3'b010)
      result = a & b;  // and
   else if (select == 3'b011)
      result = a | b;  // or
   else if (select == 3'b100)
      result = ~a;     // not
```

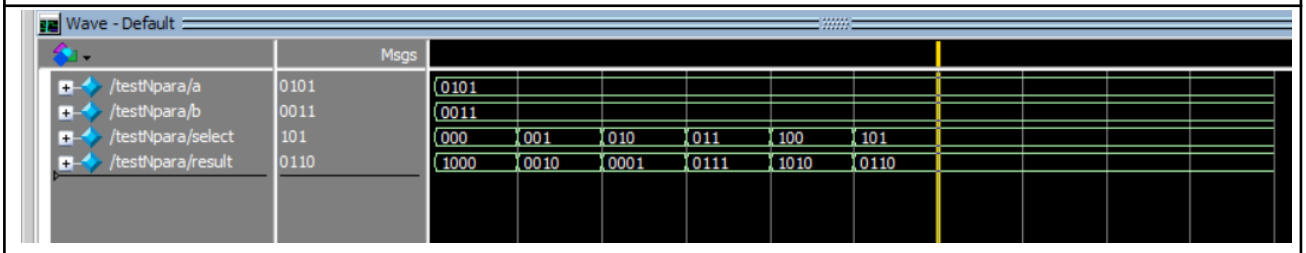
```
   else if (select == 3'b101)
      result = a ^ b;  // xor
   else
      result = 0;
end

endmodule

module testNpara();
reg [3:0] a, b;
reg [2:0] select;
wire [3:0] result;

aluParameter #(4) uut(a, b, select, result);
initial
begin
a = 5; b = 3;
select = 3'b000; #10;
select = 3'b001; #10;
select = 3'b010; #10;
select = 3'b011; #10;
select = 3'b100; #10;
select = 3'b101; #10;
end
endmodule
```

Output:



Task 4:

Description:

32-bit ALU was made that performs addition, subtraction, AND, OR, XOR, comparison, logical left shift, logical right shift and arithmetic right shift.

Code:

```
module alu32(
   input [31:0] a,
   input [31:0] b,
   input [3:0] alu_op,
   output reg [31:0] result,
   output zero
);

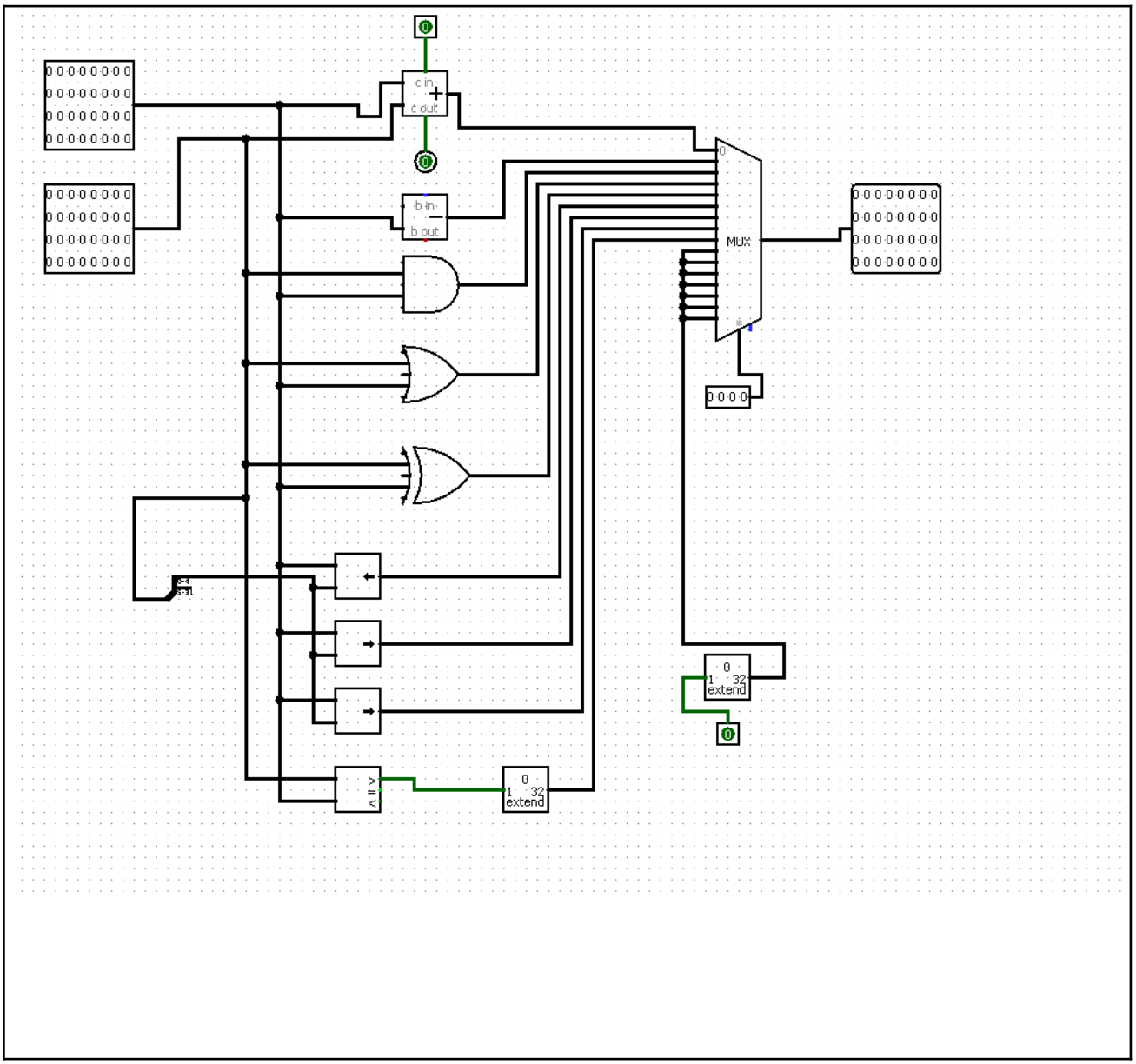
always @*
begin
   if (alu_op == 4'b0000)
      result = a + b;              // add
   else if (alu_op == 4'b0001)
      result = a - b;              // sub
   else if (alu_op == 4'b0010)
      result = a & b;               // and
   else if (alu_op == 4'b0011)
      result = a | b;              // or
   else if (alu_op == 4'b0100)
      result = a ^ b;              // xor
   else if (alu_op == 4'b0101)
      result = ($signed(a) < $signed(b)) ? 32'd1 : 32'd0; // SLT
   else if (alu_op == 4'b0110)
      result = (a < b) ? 32'd1 : 32'd0; // sltu
   else if (alu_op == 4'b0111)
      result = a << b[4:0];          // sll
   else if (alu_op == 4'b1000)
      result = a >> b[4:0];          // srl
   else if (alu_op == 4'b1001)
      result = $signed(a) >>> b[4:0];  // sra
   else
      result = 32'd0;
end
```

```
assign zero = (result == 32'd0);

endmodule
```

Task 5:

Description:
The 32 Bit ALU is created in logisim with addition,subtraction, AND,OR,XOR, comparison and shifting.
Output:


c in
c out
b in
b out
MUX
0000
0
1 32
extend
0
1 32
extend
>
=
<